

Retrieval-Augmented Generation for Research Papers

Robert Trifan

University of Bucharest

Abstract

Large language models often struggle with domain-specific information and may hallucinate facts when answering specialized queries. We present a comprehensive evaluation of retrieval-augmented generation (RAG) system components for scientific question answering, using a corpus of 10 research papers focused on BERT linguistic analysis. Our modular architecture enables systematic comparison of three chunking strategies (Sliding Window, Sentence-based, Semantic), four retrieval methods (BM25, TF-IDF, Dense with Sentence-BERT, and RRF hybrid), and multiple answer generation approaches (SimpleRAG, RAG Fusion with query expansion). The system implements a three-way classification scheme (ANSWERABLE, NOT_IN_CORPUS, UNRELATED) using an LLM-based acceptance engine with source attribution. Through quantitative evaluation on test queries, we identify the most effective component combinations and compare against baseline approaches including standalone LLMs and simple information retrieval. Our findings provide insights into the design choices that matter most for building robust RAG systems in specialized technical domains.

1 Introduction

Large language models (LLMs) have shown impressive results on various natural language processing (NLP) tasks, but they often struggle to access domain-specific information and may generate incorrect or hallucinated facts when answering questions. In this paper, we are exploring the use of retrieval-augmented generation (RAG) techniques to enhance the performance of LLMs on research question answering tasks based on a corpus of scientific papers.

RAG systems address the limitations of LLMs by enhancing their prompts with relevant retrieved documents from an external knowledge base. Their effectiveness depends critically on multiple design choices throughout the pipeline.

This work presents a comprehensive evaluation of RAG pipeline components using a corpus of 10 research papers focused on BERT linguistic analysis. We implement and compare three chunking strategies (sliding window, sentence-based and semantic chunking), four retrieval methods (TF-IDF, BM25, DENSE retrieval with Sentence Transformers, and a hybrid approach DENSE+BM25 using Reciprocal Rank Fusion) and additional postprocessing tricks such as query expansion and LLM-based answer acceptance. Our evaluation focuses on RAG’s ability to classify a query as ANSWERABLE, NOT_IN_CORPUS or UNRELATED and to provide accurate answers (with sources) for the answerable queries.

2 Related Work

2.1 Domain Adaptation for Large Language Models

Adapting large language models to domain-specific tasks can be approached through two primary paradigms: fine-tuning and retrieval augmentation. Fine-tuning methods, such as LoRA (Low-Rank Adaptation) [4], modify the model’s parameters to specialize on domain-specific data. While LoRA achieves efficient fine-tuning through low-rank matrix decomposition, it requires labeled training data, computational resources for the fine-tuning process, and must be retrained when the domain knowledge changes. Additionally, fine-tuned models lack interpretability as they do not provide explicit sources for their outputs.

In contrast, retrieval-augmented generation (RAG), introduced by Lewis et al. [7], enhances

LLM performance by retrieving relevant documents from an external knowledge base and incorporating them into the generation context. RAG offers several advantages: it requires no fine-tuning (only document indexing), can be updated by simply modifying the corpus, provides source attribution for answers, and maintains knowledge freshness. However, RAG incurs higher inference costs due to the retrieval step and increased token count in the prompt. For rapidly evolving domains like scientific research, where new papers are continuously published and explainability is crucial, RAG presents a more practical solution.

2.2 Retrieval Methods for RAG

Effective retrieval is fundamental to RAG performance. Traditional lexical methods like BM25 and TF-IDF have been widely used in information retrieval for their efficiency and interpretability. Dense retrieval methods, pioneered by Dense Passage Retrieval (DPR) [6], use neural encoders to map queries and documents into a shared embedding space, enabling semantic matching beyond keyword overlap. Implementations often leverage FAISS [5] for efficient similarity search at scale.

Recent work has explored hybrid approaches that combine lexical and semantic retrieval. Reciprocal Rank Fusion (RRF) [1] provides a theoretically grounded method for merging multiple ranked lists, demonstrating superior performance compared to individual ranking methods. Our work implements RRF to combine BM25 and dense retrieval, capitalizing on the complementary strengths of lexical precision and semantic understanding.

2.3 Advanced RAG Techniques

Beyond basic retrieval, several techniques have been proposed to enhance RAG systems. RAG-Fusion [8] introduces query expansion through multiple reformulated queries, which are then fused using reciprocal rank fusion to improve retrieval diversity and coverage. Gao et al. [3] provide a comprehensive survey of RAG advancements, highlighting the importance of chunking strategies, retrieval quality, and answer verification.

Our work contributes a systematic evaluation of these components—chunking strategies, retrieval methods, and acceptance criteria—specifically for scientific question answering, with particular emphasis on the system’s ability to appropriately refuse unanswerable queries.

3 Method

We describe in this section the design choices we made for each component of our RAG pipeline, starting from dataset preparation to retrieval and answer generation.

3.1 Dataset

Our corpus consists of 10 research papers mainly focused on the linguistic analysis of Bidirectional Encoder Representations from Transformers (BERT) [2] also called BERTology. The papers analyze BERT’s ability to encode various linguistic phenomena, including morphology, syntax, and semantics, across different layers and architectures. While the precise focus is not uniform across all papers, they collectively contribute to understanding how BERT and its variants represent linguistic information.

Figure 1 presents the distribution of paper sizes in our corpus, measured both by file size (in megabytes) and total character count. The corpus exhibits variability in paper length, with file sizes ranging from approximately 0.3 to 0.8 MB and character counts spanning from 20,000 to 90,000 characters. This diversity in paper lengths ensures that our RAG system is evaluated across documents of varying complexity and information density.

3.2 Preprocessing

3.2.1 PDF Extraction and Text Cleaning

We extract text from PDF files using `pymupdf4llm`, a specialized library built on `PyMuPDF` that converts PDFs to Markdown format before extracting text. This approach provides cleaner output compared to raw text extraction, automatically removing common

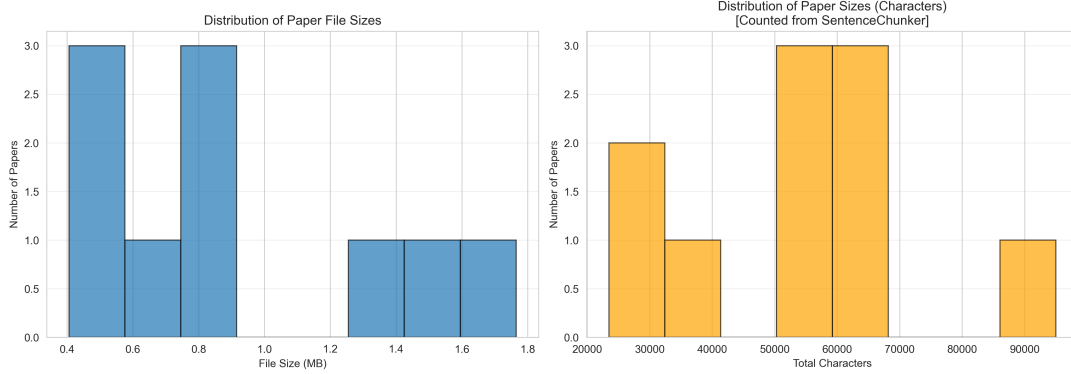


Figure 1: Distribution of paper sizes in the corpus. Left: File sizes in megabytes. Right: Total character counts computed from SentenceChunker (non-overlapping chunks).

artifacts such as page numbers, headers, footers, and figure axis labels that typically appear in academic papers. After extraction, we apply basic text normalization by collapsing whitespace sequences. To filter noise, we discard chunks with fewer than 50 characters, removing fragments such as isolated punctuation, figure labels, or incomplete sentences that provide little semantic value for retrieval.

3.2.2 Chunking Strategies

Document chunking impacts retrieval performance by determining the granularity of information units. We implement and compare three chunking strategies:

Sliding Window Chunking employs a fixed-size window (1024 characters) with 10% overlap (102 characters) that slides across the text. This approach guarantees uniform chunk sizes and ensures that information spanning chunk boundaries is captured in multiple chunks. However, it may split sentences mid-word and does not respect semantic boundaries. This strategy produced 661 chunks with an average size of 930 characters.

Sentence-Based Chunking uses NLTK’s sentence tokenizer to identify sentence boundaries and groups consecutive sentences until reaching approximately 1024 characters. When a single sentence exceeds the maximum chunk size, it is preserved intact to maintain linguistic coherence. This method respects natural language structure, producing 651 chunks with an average size of 863 characters and a maximum of 2224 characters for particularly long academic sentences.

Semantic Chunking extends sentence-based chunking by computing semantic similarity between consecutive sentences using Sentence-BERT embeddings (all-MiniLM-L6-v2). Chunks are split when cosine similarity falls below 0.1, indicating a major topic transition. This approach aims to group semantically related sentences regardless of length, though the 1024-character limit is still enforced for practical constraints. Semantic chunking resulted in 1422 chunks with an average size of 388 characters, reflecting more frequent splits at semantic boundaries.

Table 1 summarizes the statistics for each chunking strategy. The higher chunk count for semantic chunking reflects its sensitivity to topic shifts, while the comparable counts between sliding window and sentence-based chunking demonstrate that natural sentence boundaries align reasonably well with fixed-size windows in academic text. Figure 2 visualizes these differences, showing the distribution of chunk sizes across strategies.

Strategy	Chunks	Avg	Max
Sliding Window	661	930	1024
Sentence	651	863	2224
Semantic	1422	388	2224

Table 1: Chunking strategy statistics (sizes in characters).

3.2.3 Lexical Search Preprocessing

For BM25 and TF-IDF retrieval methods, we apply additional text preprocessing to improve

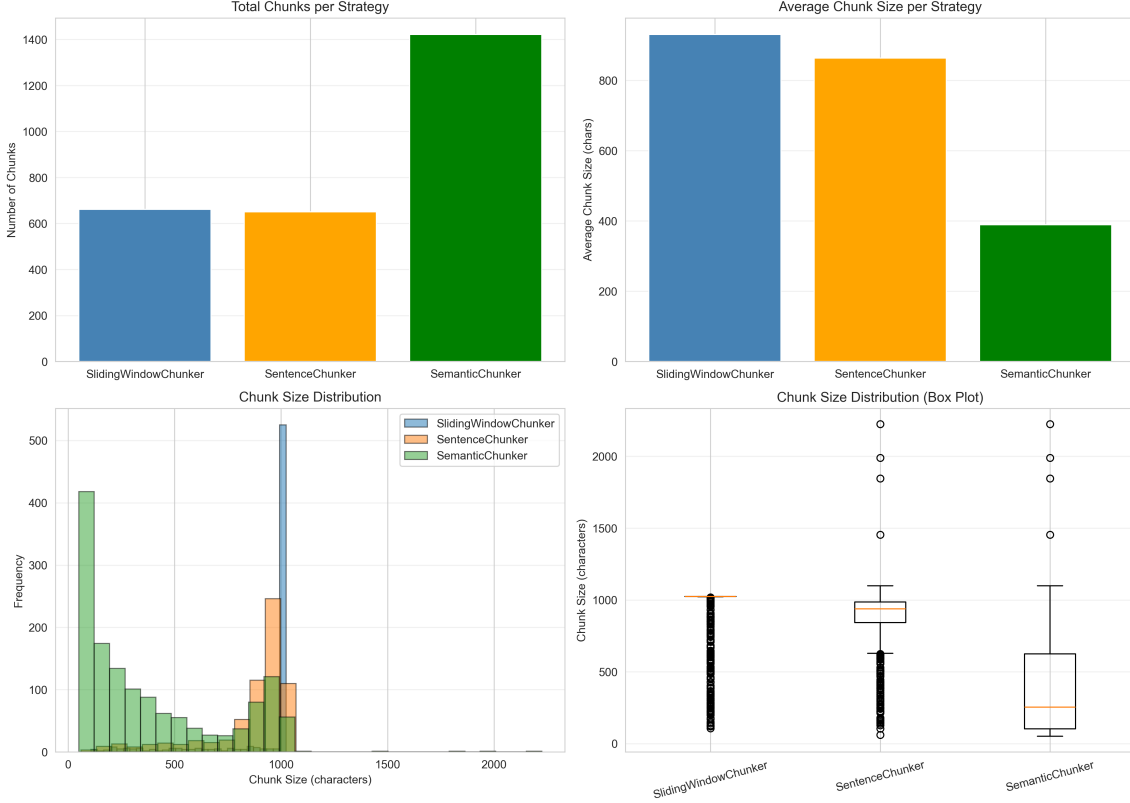


Figure 2: Comparison of chunking strategies. Top left: Total number of chunks produced by each strategy. Top right: Average chunk size. Bottom left: Overlaid histograms showing chunk size distributions. Bottom right: Box plots illustrating the spread and outliers in chunk sizes.

lexical matching. We use NLTK’s Porter Stemmer to reduce words to their root forms (e.g., "running" → "run") and remove English stop words (e.g., "the", "is", "and"). This preprocessing reduces vocabulary size and focuses matching on content-bearing terms, which empirically improves retrieval quality for these lexical methods. Dense retrieval methods (using Sentence-BERT) do not require stemming or stop word removal, as the neural encoder implicitly learns semantic representations that are robust to morphological variations.

3.3 Method and Results

3.3.1 RAG System Architecture

Our RAG system follows a modular pipeline architecture (Figure 3) that allows systematic evaluation of different component combinations. The pipeline consists of four main stages: document ingestion and preprocessing, indexing with multiple chunking strategies, retrieval with various search methods, and answer gen-

eration with acceptance criteria.

3.3.2 Retrieval Methods

We implemented four retrieval methods to enable comprehensive comparison between lexical and semantic approaches.

BM25 and TF-IDF Baselines. We use BM25 (Best Matching 25) and TF-IDF (Term Frequency-Inverse Document Frequency) as lexical baseline methods. Both implementations use custom tokenization with Porter stemming and stopwords removal to normalize query and document representations. BM25 ranks documents using the BM25Okapi implementation, while TF-IDF uses scikit-learn’s TfidfVectorizer with cosine similarity for scoring.

Dense Retrieval. For semantic search, we implement dense retrieval using Sentence-BERT [9]. We encode all text chunks using the `all-MiniLM-L6-v2` model to obtain 384-dimensional embeddings. For efficient similarity search over the vector representations, we use FAISS [5] with the `IndexFlatIP` index,

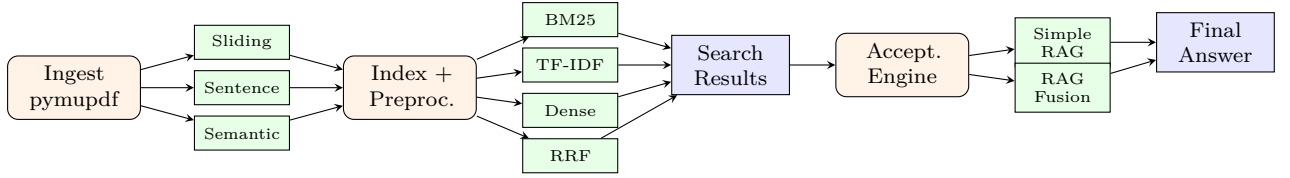


Figure 3: RAG system pipeline architecture showing modular components. The system supports 3 chunking strategies, 4 retrieval methods, and 2 generation approaches, enabling systematic evaluation of 24 different configurations.

which performs exact inner product search. This configuration is suitable for our corpus size and provides accurate retrieval without approximation.

Reciprocal Rank Fusion (RRF). To combine the strengths of lexical and semantic retrieval, we implement RRF [1] as a reranking strategy. RRF merges results from BM25 and Dense search by computing reciprocal rank scores for each retrieved chunk and aggregating them. This hybrid approach leverages both keyword matching and semantic similarity to improve retrieval quality.

3.3.3 Acceptance Criteria

To filter low-quality or irrelevant retrieval results before answer generation, we implement an LLM-based acceptance engine that performs three-way classification of each query:

- **ANSWERABLE:** The retrieved sources explicitly contain sufficient information to answer the query.
- **NOT_IN_CORPUS:** The query is within the corpus domain (BERT, bidirectional language models, linguistic probing, etc.), but the retrieved sources do not contain the answer.
- **UNRELATED:** The query is outside the corpus domain entirely.

The acceptance process operates in two stages. First, we retrieve the top-k most relevant chunks for the query using the configured search method. Then, we pass these retrieved chunks to the LLM for classification. To enable fine-grained source attribution, we prefix each retrieved chunk with its unique identifier in square brackets (e.g., [chunk_id]). The LLM is instructed through a structured prompt

(see Appendix) to examine only these retrieved chunks and reference the identifiers when classifying a query as ANSWERABLE, allowing us to track which specific chunks support the generated answer. The system validates that all returned chunk IDs correspond to actual retrieved chunks and enforces that ANSWERABLE decisions must include at least one supporting chunk ID, while non-ANSWERABLE decisions must have empty chunk ID lists.

3.3.4 Answer Generation

We implement two answer generation approaches to compare standard and advanced RAG architectures:

SimpleRAG. Our baseline follows the standard RAG pipeline [7]: retrieve top-k chunks, evaluate acceptance criteria, and generate an answer only if the query is classified as ANSWERABLE. For answerable queries, we concatenate the retrieved chunks as context and prompt the LLM to generate a response. For non-answerable queries (NOT_IN_CORPUS or UNRELATED), the system returns no answer along with the classification decision.

RAG Fusion. Following [8], we implement query expansion to improve retrieval diversity. The system uses an LLM to generate multiple paraphrased variants of the original query (see Appendix for the expansion prompt). The number of query variations is a configurable parameter that can be adjusted based on the trade-off between retrieval coverage and computational cost. Each variant is independently processed through the retrieval pipeline, and the results are combined using RRF reranking. This expanded result set aims to extract more comprehensive context by capturing relevant chunks from different semantic perspectives. The fused results are then evaluated by the acceptance engine and processed through an-

answer generation using the same approach as SimpleRAG. This architecture mitigates the vocabulary mismatch problem and improves recall for queries where a single formulation might miss relevant information.

Baseline Comparisons. To evaluate the contribution of each RAG component, we implement two baseline approaches. First, a *simple information retrieval* system that performs retrieval without answer generation, returning only the top-k retrieved chunks using TF-IDF. This baseline isolates retrieval quality from generation quality. Second, we implement a *standalone LLM* approach using TinyLlama through Ollama, which answers queries using only its parametric knowledge without any retrieval. This baseline measures the value added by the retrieval augmentation compared to pure LLM capabilities. Comparing these baselines against SimpleRAG and RAG Fusion allows us to quantify the impact of retrieval augmentation and query expansion on answer quality.

3.3.5 Evaluation Results

We conduct quantitative evaluation using 10 test queries from our evaluation set, which includes answerable, not-in-corpus, and unrelated questions. We measure classification accuracy by comparing the system’s acceptance decision (ANSWERABLE, NOT_IN_CORPUS, or UNRELATED) against the ground truth query type. All experiments use SimpleRAG with the LLM-based acceptance engine.

Chunking Strategy Comparison We evaluate the three chunking strategies using Dense retrieval (FAISS IndexFlatIP). Table 2 presents the classification accuracy for each strategy.

Strategy	Correct	Total	Accuracy
Sliding Window	10	10	100.0%
Sentence	10	10	100.0%
Semantic	9	10	90.0%

Table 2: Chunking strategy comparison with SimpleRAG + Dense retrieval.

Both Sliding Window and Sentence-based chunking achieved perfect classification accuracy (100%), correctly identifying all answer-

able (7/7), not-in-corpus (2/2), and unrelated (1/1) queries. Semantic chunking achieved slightly lower accuracy (90%) due to one misclassification in the answerable category (6/7 correct). This suggests that while semantic chunking creates more coherent topic-based chunks, the finer granularity (1422 smaller chunks vs 661/651 larger chunks) may occasionally fragment important context needed for accurate query classification. The superior performance of sentence-based and sliding window approaches indicates that preserving longer contiguous text spans is beneficial for the acceptance engine’s decision-making process.

Retrieval Method Comparison We compare the four retrieval methods using Sliding Window chunking. Table 3 shows the classification accuracy for each method.

Method	Correct	Total	Accuracy
BM25	10	10	100.0%
TF-IDF	10	10	100.0%
Dense	10	10	100.0%
RRF	10	10	100.0%

Table 3: Retrieval method comparison with SimpleRAG + Sliding Window chunking.

Remarkably, all retrieval methods achieved perfect classification accuracy (100%) on our evaluation set. This result demonstrates that with Top-K=5 retrieval and a sufficiently capable LLM acceptance engine (gpt-5-nano), the retrieval strategy has minimal impact on classification performance for our BERT-focused corpus. Both keyword-based methods (BM25, TF-IDF with Porter stemming and stopword removal) and semantic methods (Dense retrieval with Sentence-BERT embeddings) successfully retrieved relevant context. The RRF hybrid approach, combining lexical and semantic signals through reciprocal rank fusion, maintained this perfect accuracy. This suggests that for a focused technical corpus with 10 papers, the retrieval quality is consistently high across different paradigms, and the LLM’s ability to reason about retrieved context is the primary factor in accurate query classification.

Answer Generation Approach Comparison We compare the four assistant types

on an answerable query: *"How do BERT's internal representations evolve across layers?"*

SimpleRAG:

Decision: ANSWERABLE

Answer: BERT's representations are organized hierarchically along the layer stack. Lower layers tend to encode more lexical/syntactic information, while higher layers accumulate more abstract, task-relevant knowledge (semantics/world knowledge).

Sources: [2004.14448v1] Merchant et al.; [1905.05950v2] Tenney et al.; [2002.12327v3] Rogers et al.

RAG Fusion:

Decision: ANSWERABLE

Answer: Overall pattern: BERT's representations evolve from input-like, low-level features in the lower layers to more abstract, task-agnostic features in the early-to-middle layers, and finally to more task-specific representations in the upper layers used by the classifier.

Sources: [1905.05950v2] Tenney et al.; [2002.12327v3] Rogers et al.

LocalLLM (TinyLlama):

Answer: BERT (Bayesian Ensemble Reconstruction Toolkit) is a probabilistic modeling tool that utilizes Bayesian inference to capture the relationship between input features and outputs. The internal representations of BERT are formed by multiple hidden layers in the network. Each layer generates probabilities based on the input features, which are then combined in a way that maximizes the likelihood of output tokens given the input features.

Note: No retrieval; answer lacks specific technical details from the corpus.

SimpleInfoRetrieval:

Retrieved Papers: [2002.12327v3] Rogers et al.; [1808.08949v2] Peters et al.; [2004.14448v1] Merchant et al.; [1802.05365v2] Peters et al.

Note: Returns raw chunks (along with their sources) without synthesis; requires manual reading to extract answer.

Both SimpleRAG and RAG Fusion correctly identify the query as answerable and provide accurate, corpus-grounded responses with source attribution. RAG Fusion retrieved 2 highly relevant papers, while SimpleRAG consulted 3 papers for a more comprehensive answer. LocalLLM without retrieval produces

a generic answer lacking the specific technical contributions documented in the papers, demonstrating the value of retrieval augmentation. SimpleInfoRetrieval successfully retrieved 4 relevant papers (including additional ELMO contextualization work by Peters et al.) but provides no synthesis, placing the burden of interpretation entirely on the user.

4 Future Work

Several promising directions could enhance the RAG system's capabilities:

BERT-based Chunk Relevance Classification Instead of relying solely on the LLM acceptance engine, we could fine-tune a BERT model for question answering to classify each retrieved chunk's relevance. By performing inference on (question, chunk) pairs, the model could directly predict ANSWERABLE, NOT_IN_CORPUS, or UNRELATED categories, potentially reducing API costs and improving consistency.

Hierarchical Context Management

When retrieved chunks exceed the LLM's context window, a voting or ensemble approach could be employed. Chunks could be grouped into batches, each processed independently, with final decisions aggregated through majority voting or confidence-weighted fusion. This would enable scaling to larger Top-K values without context limitations.

Advanced Query Augmentation While RAG Fusion demonstrates the value of query expansion, more sophisticated techniques could further improve retrieval. Chain-of-thought prompting could help the LLM decompose complex queries into sub-questions, while few-shot examples could guide query reformulation. Iterative refinement based on initial retrieval results could also identify knowledge gaps.

5 Conclusion

We presented a comprehensive RAG system for scientific question answering on a corpus of

10 BERT-focused papers, systematically evaluating three chunking strategies, four retrieval methods, and multiple answer generation approaches.

Our experiments show that Sliding Window and Sentence-based chunking achieved perfect classification accuracy (100%), outperforming Semantic chunking (90%). Remarkably, all retrieval methods—BM25, TF-IDF, Dense, and RRF—achieved 100% accuracy, indicating that for a focused technical corpus, the LLM’s reasoning over retrieved context is more critical than the retrieval strategy itself. Comparison of generation approaches confirmed that retrieval augmentation is essential: SimpleRAG and RAG Fusion produced accurate, corpus-grounded answers, while the standalone TinyLLM baseline generated generic responses lacking specific technical details.

These results validate the effectiveness of modular RAG architectures for specialized technical domains, where preprocessing quality and LLM-based acceptance engines enable robust question classification across diverse retrieval strategies.

References

- [1] Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *SIGIR '09: Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pages 758–759, New York, NY, USA, 2009.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [3] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey, 2024.
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [5] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus, 2017.
- [6] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. Dense passage retrieval for open-domain question answering, 2020.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
- [8] Zackary Rackauckas. Rag-fusion: A new take on retrieval augmented generation. *International Journal on Natural Language Computing*, 13(1):37–47, February 2024.
- [9] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019.

A LLM Acceptance Prompt

The following system prompt is used to classify queries in the acceptance engine:

You are a strict classifier for a Retrieval-Augmented Generation (RAG) system.

Classify the user's question into exactly one of:

- ANSWERABLE: The provided sources explicitly contain enough information to answer.
- NOT_IN_CORPUS: The question is within the corpus domain (BERT/bidirectional LMs, layers, probing, attention heads, morphology/syntax/semantics, representation analysis), but the sources do NOT contain the answer.
- UNRELATED: The question is outside that domain.

Rules:

- Each chunk is prefixed with [chunk_id]. Reference these IDs in supporting_chunk_ids.
- Use ONLY the provided sources to decide ANSWERABLE vs NOT_IN_CORPUS.
- If the answer is not explicitly supported, do NOT label ANSWERABLE.
- If decision=ANSWERABLE, supporting_chunk_ids MUST contain at least one chunk_id from the sources.
- If decision!=ANSWERABLE, supporting_chunk_ids MUST be empty.
- Provide a short reason (1-2 sentences).

B Query Expansion Prompt

The following prompts are used for query expansion in RAG Fusion:

System Prompt:

You are an expert at query reformulation for information retrieval. Generate alternative phrasings that maintain semantic equivalence.

User Prompt:

Be creative while rephrasing the following question while preserving its meaning and helping a retrieval system find relevant documents.

Return ONLY the rephrased question without any explanation or additional text.

Original question: {query}

Rephrased question: